

Programming Assignment 4.2

Deepesh Devang Parpani : 35230862

Coordination server (Zookeeper): A coordination protocol using Zookeeper as a logically centralized service accessible to all replicas;

This implementation manually implements the consensus mechanism by using Zookeeper as the shared log.

We essentially use `PERSISTENT_SEQUENTIAL` flag to automatically number the requests and ensure total ordering.

As the log is simply just a list of znodes in a zookeeper directory, all the servers watch this directory and when a new file appears they execute the command inside it in their local db.

Code Design:

- **Constructor logic:**
Init zookeeper and define the directory structure for requests and checkpoint.
It calls the `performrecovery` method to ensure if the server has crashed or missed data using checkpoint and then replays the state from there onwards.
- **handleMessageFromClient:**
Handles all incoming messages from testclient. It doesn't execute the query immediately but it first writes the request to zookeeper and zookeeper assigns it a unique number as a part of proposal phase. The server essentially waits until its own watcher sees the file and then executes it and client only receives the ack when data is pushed into actual db.
- **Replicated State Machine logic:**
We have a watch method that fires whenever a child node is added to the requests directory.
We then have the `processNewRequests` method to get the list of requests and sort them.

Then we apply CQL to cassandra using execute request.

- Checkpointing logic:
As the max log size is 400, if we don't delete old requests zookeeper might crash and hence we use the logic to trigger checkpointing when the number of requests executed since last save exceeds at least 200.
We then dump the current cassandra state to a json string in zookeeper and then clean up the old requests.
- State Transfer or Recovery logic:
This handles the scenario where the server has been dead for a long time. This uses the checkpoint or snapshot when the old requests are deleted and from that point we can replay the requests to catch up to the current state and remain consistent.

Flow of code:

- 1) Init
- 2) Restore JSON
- 3) Replay log
- 4) Watch Zookeeper
- 5) Handle client write in Thread A
 - a) Receive
 - b) Write ZK Node
 - c) Wait and then ack
- 6) Handle watcher in Thread B:
 - a) Sort nodes
 - b) Execute new requests
 - c) Unblock thread A
- 7) Maintenance log:
 - a) Dump db to json
 - b) Then delete old znodes.